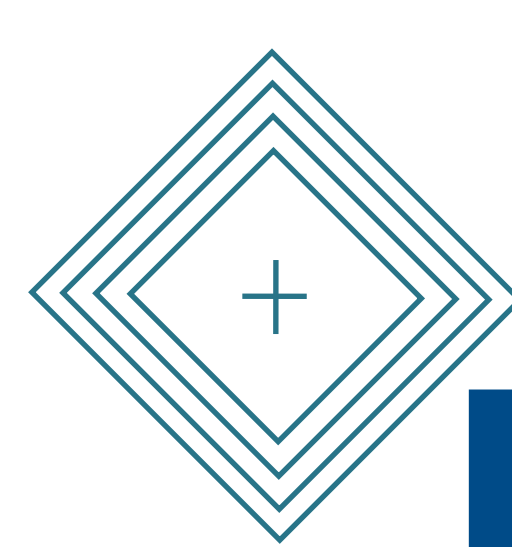


Basic Analysis and Algorithmic Methods

JAHEZIA

**College of Science
Northern Border University**





Best Case

1. The algorithm performs **minimum number of operations**.
2. Represents the **most favorable input** for the algorithm.
3. Example: In linear search, if the target element is the **first element**, time complexity is **$O(1)$** .



Average Case

1. Expected performance for **random or typical input data**.
2. Calculated by considering all possible inputs and their probabilities.
3. Example: In linear search, average complexity is **$O(n/2) \approx O(n)$** .



Worst Case

1. The algorithm performs **maximum number of operations**.
2. Represents the **least favorable input** or most time-consuming scenario.
3. Example: In linear search, if the element is **not present** or is the **last element**, complexity is **$O(n)$** .



Case Behaviors of an Algorithm

When analyzing an algorithm, we evaluate its performance in **different input scenarios**. The three main cases describe how the algorithm behaves under varying conditions:

- **Best Case: Big Omega (Ω)** notation, which provides a **lower bound** on the algorithm's running time.
- **Average Case: Big Theta (Θ)** notation, which provides a tight bound (**both an upper and lower bound**) on the algorithm's running time.
- **Worst Case: Big O (O)** notation, which provides an **upper bound** on the algorithm's running time.

Big O Notation and Complexity of Algorithms

What is Big O Notation?

- **Big O notation** is a mathematical representation that describes **how an algorithm's runtime or space requirements grow** with the size of the input (n).
- It helps compare **algorithm efficiency** regardless of hardware or programming language.
- Expresses the **upper bound** of growth rate — i.e., **worst-case performance**.

COMPLEXITY	DESCRIPTION	EXAMPLE ALGORITHMS
$O(1)$	CONSTANT TIME	ACCESSING AN ARRAY ELEMENT
$O(\log N)$	LOGARITHMIC	BINARY SEARCH
$O(N)$	LINEAR	LINEAR SEARCH
$O(N \log N)$	LOG-LINEAR	MERGE SORT, QUICK SORT
$O(N^2)$	QUADRATIC	BUBBLE SORT, INSERTION SORT
$O(2^n)$	EXPONENTIAL	RECURSIVE FIBONACCI
$O(N!)$	FACTORIAL	TRAVELING SALESMAN (BRUTE FORCE)

Search Algorithms

1. Linear Search

Concept: Sequentially checks each element in a list until the target is found or the list ends.

Best Case: $O(1)$ (target is first element)

Worst Case: $O(n)$

Space Complexity: $O(1)$

2. Binary Search

Concept: Works on sorted arrays; repeatedly divides the search interval in half.

Best Case: $O(1)$

Average/Worst Case: $O(\log n)$

Space Complexity: $O(1)$ (iterative) or $O(\log n)$ (recursive)

Sorting Algorithms

3. Bubble Sort

Concept: Repeatedly swaps adjacent elements if they are in the wrong order.

Best Case: $O(n)$ (already sorted)

Average/Worst Case: $O(n^2)$

Space Complexity: $O(1)$

4. Selection Sort

Concept: Repeatedly finds the smallest element from the unsorted part and moves it to the sorted part.

Best/Worst/Average Case: $O(n^2)$

Space Complexity: $O(1)$

5. Insertion Sort

Concept: Builds a sorted list one element at a time by inserting elements into their correct position.

Best Case: $O(n)$

Average/Worst Case: $O(n^2)$

Space Complexity: $O(1)$

6. Merge Sort

Concept: Divides the array into halves, recursively sorts them, and merges the sorted halves.

Best/Average/Worst Case: $O(n \log n)$

Space Complexity: $O(n)$

Standard Complexity Classes.

Standard complexity classes, like P and NP, are sets of computational problems grouped by the resources (typically time or space) needed to solve them, as defined by a specific model of computation.

Class P includes problems solvable in polynomial time, which are considered "efficient," while class NP includes problems where a potential solution can be verified in polynomial time. These classes form a central part of computational complexity theory.

Common Complexity Classes

P: Problems solvable by a deterministic algorithm in polynomial time — considered efficiently solvable.

Example: Sorting, Searching.

NP: Problems for which a “yes” answer can be verified in polynomial time (even if finding it is hard).

Example: Sudoku, Traveling Salesman (decision form).

CoNP: Problems where a “no” answer can be verified in polynomial time — the complement of NP.

NP-Hard: Problems at least as hard as the hardest in NP; any NP problem can be reduced to them in polynomial time.

Example: Halting Problem, Optimization TSP.

NP-Complete: Problems that are both NP and NP-Hard — the hardest problems in NP.

Example: Boolean Satisfiability (SAT), Hamiltonian Cycle.

Algorithm Performance

Algorithm performance is the **evaluation of how efficiently an algorithm uses computational resources like time and memory**. It is measured using **time complexity** (execution time) and **space complexity** (memory usage) to compare algorithms and predict their performance, especially as input size grows. Key components of performance evaluation include defining metrics, using benchmarks, and employing diverse test data.

Common Complexity Classes

Evaluating Algorithm Performance

- **Analyze:** Study the algorithm to estimate **time** and **space complexity**.
- **Measure:** Implement and test on various input sizes to gather **empirical data**.
- **Benchmark:** Compare results with **standard algorithms** solving the same problem.
- **Optimize:** Identify **bottlenecks** and improve efficiency based on analysis.

Why algorithm performance matters

- **Efficiency:** An efficient algorithm uses minimal resources, which is critical for applications dealing with large datasets or requiring real-time responses.
- **Scalability:** Performance analysis helps predict how an algorithm will behave as the input size increases, ensuring it can scale to handle future demands.
- **Comparison:** It provides a way to choose the most optimal algorithm for a specific task by comparing different algorithms objectively.

Greedy Algorithm

A greedy algorithm solves problems by choosing the most promising option at each step, aiming for a globally optimal solution without reconsidering previous decisions or future consequences.

Key Characteristics of Greedy Algorithms

- **Locally Optimal Choices:** Makes the best decision at each step based on current conditions.
- **No Backtracking:** Once a choice is made, it isn't revised.
- **Not Always Globally Optimal:** May fail for problems lacking specific properties.
- **Greedy Choice Property:** Global optimum achievable via local choices.
- **Optimal Substructure:** Optimal overall solution includes optimal sub problem solutions.

Key Characteristics

1. Locally Optimal Choices:

At every step, the algorithm selects the option that seems best at that instant.

2. No Backtracking:

Once a decision is made, it's not changed later.

3. Efficiency:

Usually faster and simpler than exhaustive algorithms.

4. May Not Be Globally Optimal:

The greedy choice might not always lead to the best overall (global) solution — unless the problem has the greedy-choice property and optimal substructure.

General Steps of a Greedy Algorithm

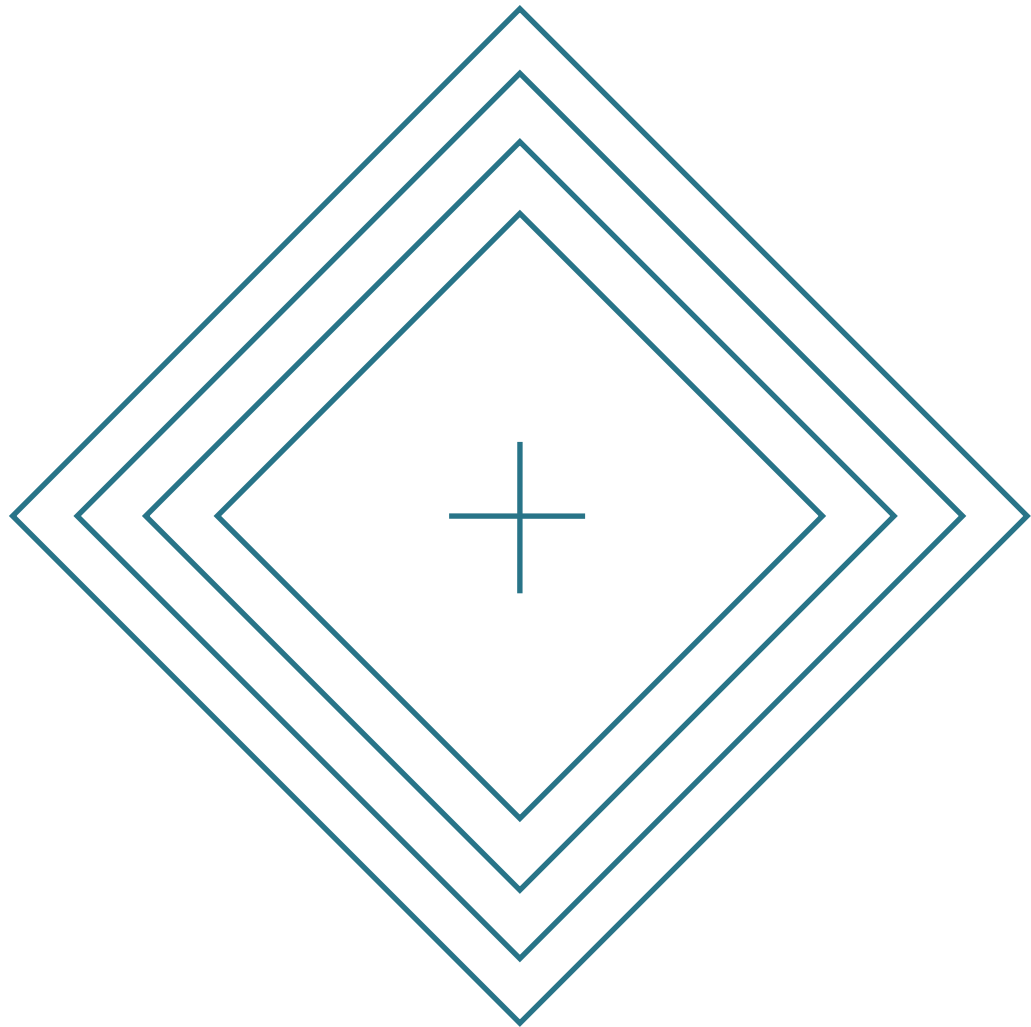
5. Initialize — Start with an empty solution.

6. Select — Choose the best available option by some criterion.

7. Feasibility Check — Ensure the choice doesn't violate constraints.

8. Include — Add this choice to the current solution.

9. Repeat until the solution is complete or no more choices remain.

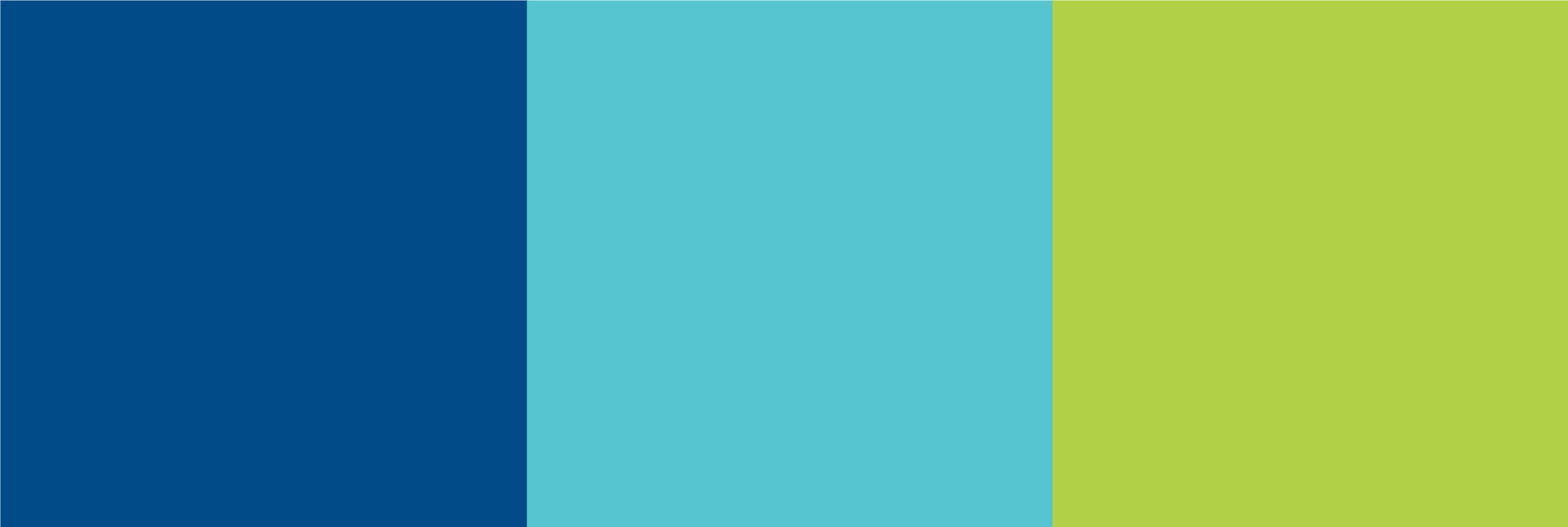


Examples of Greedy Algorithms

PROBLEM	GREEDY APPROACH	DESCRIPTION
COIN CHANGE PROBLEM	TAKE THE LARGEST COIN FIRST	WORKS WHEN DENOMINATIONS ARE CANONICAL (E.G., 1, 5, 10, 25)
KRUSKAL'S ALGORITHM	ADD SMALLEST EDGE THAT DOESN'T FORM A CYCLE	FINDS MINIMUM SPANNING TREE (MST)
PRIM'S ALGORITHM	ADD SMALLEST EDGE CONNECTED TO TREE	ANOTHER MST ALGORITHM
DIJKSTRA'S ALGORITHM	ALWAYS CHOOSE THE NEXT CLOSEST VERTEX	FINDS SHORTEST PATH IN A WEIGHTED GRAPH
HUFFMAN CODING	COMBINE LEAST FREQUENT SYMBOLS	CREATES OPTIMAL PREFIX CODES FOR DATA COMPRESSION

Working of Greedy Algorithms

WITH A GOAL OF REACHING THE LARGEST SUM, AT EACH STEP, THE GREEDY ALGORITHM WILL CHOOSE WHAT APPEARS TO BE THE OPTIMAL IMMEDIATE CHOICE, SO IT WILL CHOOSE 12 INSTEAD OF 3 AT THE SECOND STEP AND WILL NOT REACH THE BEST SOLUTION, WHICH CONTAINS 99



Divide-And-Conquer Algorithm

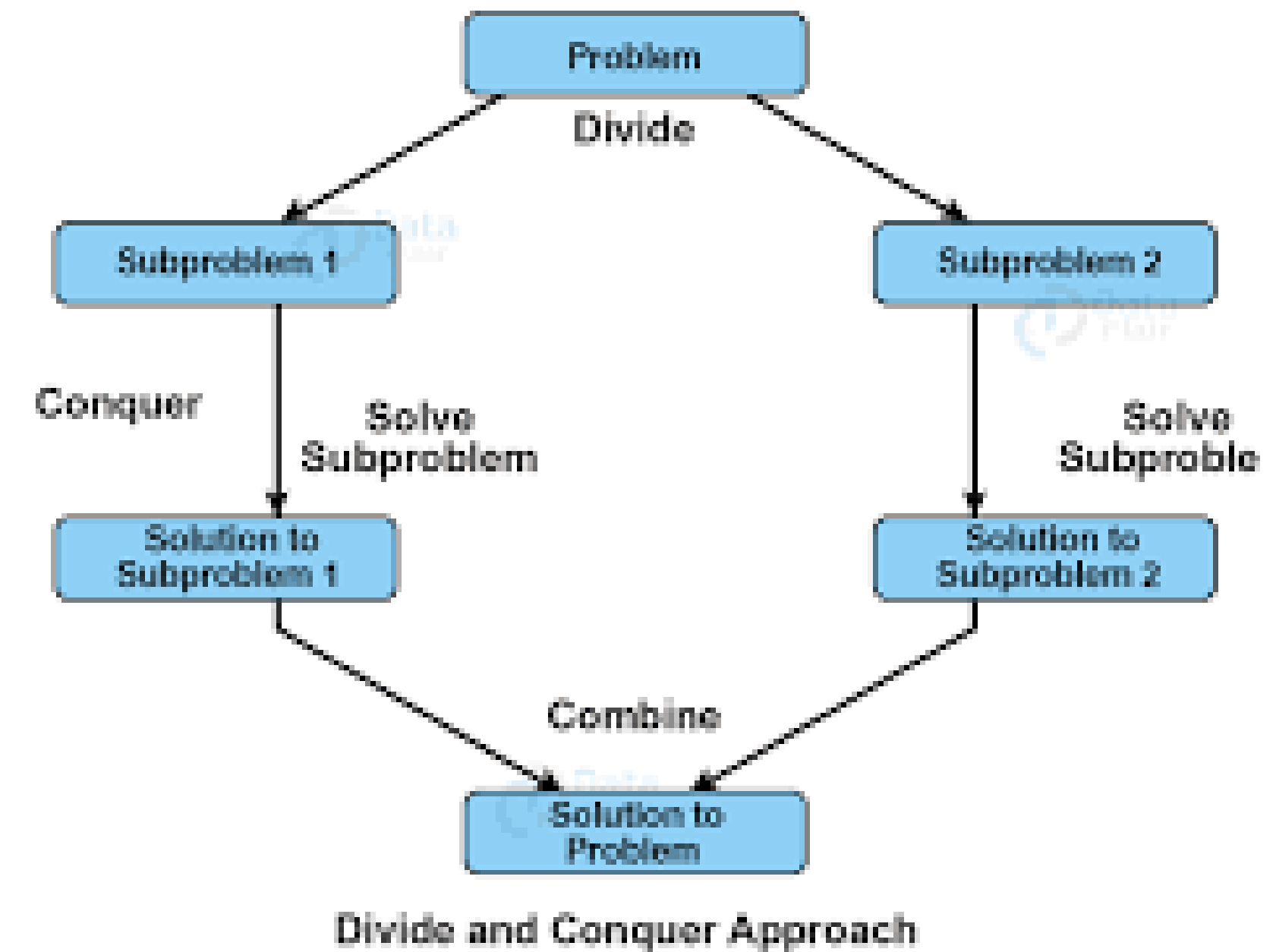
A Divide and Conquer algorithm splits a problem into smaller sub problems, solves them recursively, and merges their results for the final solution. It follows three steps: Divide, Conquer, and Combine. This approach is efficient for problems like sorting (Merge Sort, Quick Sort) and searching (Binary Search).

The three steps of a Divide and Conquer algorithm

Divide: Split the main problem into smaller, independent subproblems of the same type.

Conquer: Solve the subproblems by calling the same algorithm recursively. The recursion stops when the subproblems are small enough to be solved directly (this is called the base case).

Combine: Merge the solutions of the subproblems to form the solution to the original problem.



Examples of Divide and Conquer algorithms

- **Merge Sort:** A sorting algorithm that divides a list into halves, sorts each half recursively, and then merges them back together.
- **Quick Sort:** A sorting algorithm that picks a "pivot" element, partitions the array around the pivot, and recursively sorts the sub-arrays.
- **Binary Search:** A search algorithm that repeatedly divides the search interval in half. If the value of the search key is less than the item in the middle of the interval, the half is discarded; otherwise, the half in which the key might be found is selected.

Recursive Backtracking

Recursive backtracking explores possible solutions step-by-step using recursion. When a path fails, it backtracks to a previous choice and tries another. This method efficiently searches large solution spaces, making it ideal for problems like the n -queens puzzle, maze generation, and other combinatorial challenges with multiple possibilities.

Key Applications:

N-Queens Problem: Place n queens so none attack each other.

Maze Generation: Create random mazes using backtracking paths.

Permutations & Combinations: Generate all possible arrangements.

Puzzles (e.g., Sudoku): Fill cells under given constraints.

How it works

- **Make a choice:** At a given point, select an option and build a partial solution.
- **Recursively explore:** Call the function again for the new, smaller problem. For example, in the n -Queens problem, after placing a queen, the problem becomes placing the remaining queens on the reduced board.
- **Check for a solution:** If the current path leads to a valid, complete solution, it is recorded.
- **Backtrack:** If the path leads to a dead end (a point where no further valid choices can be made), the algorithm undoes the last choice and returns to the previous decision point to try a different option.
- **Unwind the recursion:** This process continues until the base case is reached. When a recursive call returns, the function undoes the changes it made before making the recursive call, effectively reverting the state to what it was before the call.

Dynamic Programming

Dynamic programming is a problem-solving technique that tackles complex challenges by dividing them into smaller, overlapping subproblems and storing their solutions to prevent repeated calculations. It is especially useful for optimization tasks, where the objective is to determine the most efficient or optimal solution from various possible options.

Dynamic Programming (DP) is a method for solving problems by:

- **Breaking them down** into smaller overlapping subproblems.
- **Storing the results** of these subproblems (usually in an array or table).
- **Reusing** those stored results instead of recomputing them.

This avoids redundant work and greatly improves efficiency.

Key Characteristics of Dynamic Programming Problems :

- **Overlapping Subproblems:** The problem can be broken into smaller subproblems that recur multiple times. DP optimizes by storing and reusing their results (via memoization or tabulation).
- **Optimal Substructure:** The optimal solution to the entire problem can be built from the optimal solutions of its subproblems.

Two primary approaches to dynamic programming:

- **Memoization (Top-Down):** This approach involves solving the problem recursively, but storing the results of subproblems in a cache (e.g., a hash map or array) as they are computed. When a subproblem is encountered again, its pre-computed solution is retrieved from the cache instead of re-calculating it.
- **Tabulation (Bottom-Up):** This approach involves building up the solution iteratively, starting from the smallest subproblems and progressively solving larger ones. The results are typically stored in a table (e.g., an array), and each entry in the table represents the solution to a specific subproblem.

Heuristic Approach

a heuristic approach is an algorithmic strategy that employs experience-based techniques or approximate methods to derive satisfactory or near-optimal solutions efficiently. It prioritizes computational feasibility and speed over absolute optimality, making it particularly valuable for addressing complex optimization and artificial intelligence problems where exact solutions are impractical.

Key Features:

- Focuses on **speed and efficiency** over perfection.
- Uses **experience-based strategies** or **approximations**.
- Commonly applied when problems are **NP-hard** or **combinatorial** (e.g., Traveling Salesman Problem).

How It Works :

- **Experience-based:** Uses practical knowledge or educated guesses to guide solutions.
- **Speed-focused:** Finds results faster than exhaustive algorithms.
- **Accuracy trade-off:** Produces good-enough, not necessarily optimal, solutions.
- **Complex problem use:** Ideal for large or computationally intensive problems like optimization or navigation.

Examples of Heuristic Approaches:

- **Greedy algorithms** – make the best local choice at each step (e.g., Dijkstra's algorithm).
- **Genetic algorithms** – use evolutionary principles to evolve better solutions.
- **Simulated annealing** – probabilistically explores possible solutions to escape local optima.
- **A*** search – uses a heuristic function to guide search towards the goal efficiently.